

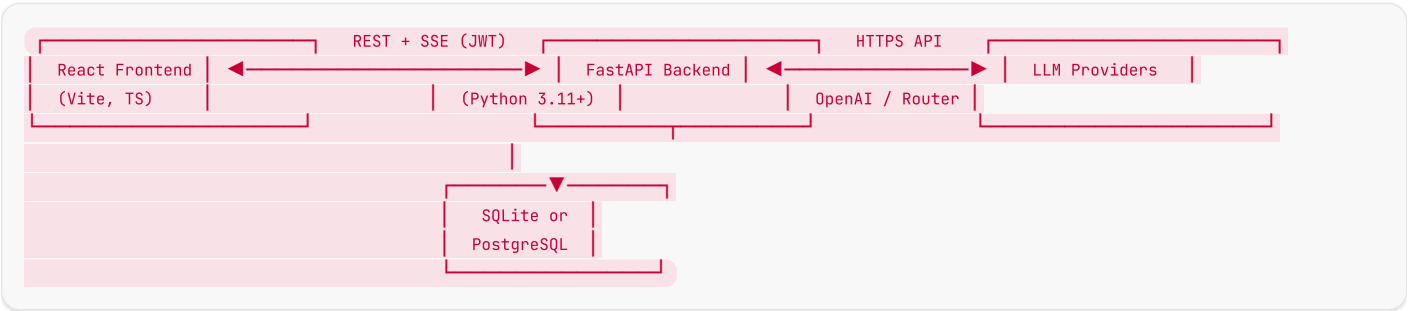
Architecture & Design

Brief explanation of how the Chatbot Platform is structured, how data flows through the system, and the design decisions behind it.

For setup and API reference, see the [main README](#).

System Overview

A multi-user web app where each user creates **projects** (AI agents) with their own LLM provider, API key, model, prompts, and chat history. The stack is a classic three-tier SPA + API:



Layer	Technology	Responsibility
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4	Auth UI, agent management, streaming chat
Backend	FastAPI, SQLAlchemy, Pydantic	REST API, JWT auth, LLM orchestration
Data	SQLite (dev) / PostgreSQL (prod)	Users, projects, messages, file metadata
External	OpenAI SDK, httpx	LLM inference and file storage

Design Principles

Multi-tenant isolation

Every resource is scoped to a **user**. Route handlers call `_get_owned_project()` (or equivalent filters) before reading or mutating data. A user never sees another user's projects, messages, prompts, or files.

Stateless API

Authentication uses **JWT Bearer tokens** (HS256, 7-day expiry by default). No server-side sessions. The API can scale horizontally without sticky sessions.

Per-agent LLM configuration

Each project stores its own provider, API key, base URL, and model. One user can run OpenAI on one agent and a local Ollama endpoint on another without global provider settings.

Provider abstraction

`backend/app/services/llm.py` centralizes provider differences behind two entry points:

- `chat_completion_with_config()` — full response
- `chat_completion_stream_with_config()` — token stream

Configuration is passed as a detached `LLMProjectConfig` dataclass so streaming handlers can work outside the original SQLAlchemy session.

Streaming first

The primary chat path is **Server-Sent Events (SSE)**. Tokens are forwarded to the client as they arrive, reducing perceived latency. The complete assistant reply is persisted only after the stream finishes.

Backend Structure

```
backend/app/
├── main.py          # App factory, CORS, router registration, DB init on startup
├── config.py        # Settings from environment (.env)
├── database.py      # SQLAlchemy engine, SessionLocal, get_db dependency
├── models.py        # ORM entities
├── schemas.py       # Pydantic request/response models
├── auth.py          # Password hashing, JWT create/decode, get_current_user
├── routes/
│   ├── auth.py      # register, login, me
│   ├── projects.py  # CRUD for agents
│   ├── prompts.py   # Named prompt templates per project
│   ├── chat.py      # Messages, chat, streaming chat
│   └── files.py      # File upload metadata (OpenAI Files API)
├── services/
│   └── llm.py        # Provider routing, instruction building, streaming
```

Routers are mounted under `/api` in `main.py`. A health check lives at `GET /api/health`.

Data Model

```
User
├── Project (1:N) ← "agent" in the UI
├── Prompt (1:N)  ← named instruction blocks
├── Message (1:N) ← chat history (user | assistant)
└── ProjectFile (1:N) ← upload metadata + OpenAI file ID
```

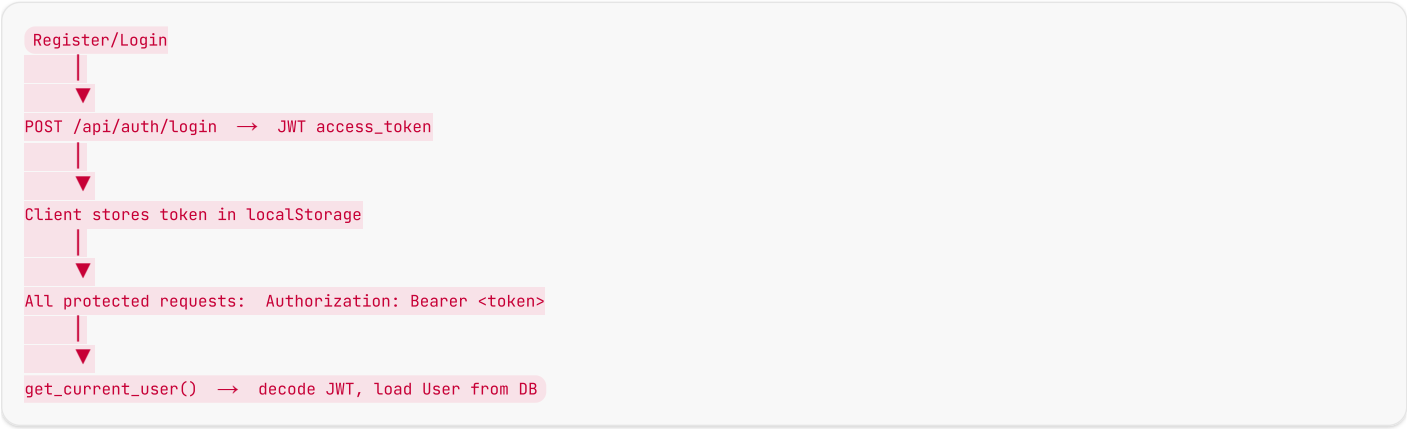
Entities

Entity	Purpose
User	Email + bcrypt-hashed password
Project	Agent name, system prompt, LLM provider (<code>openai</code> <code>openrouter</code> <code>custom</code>), API key, base URL, model
Prompt	Named content merged into LLM instructions at chat time
Message	Persisted chat turns; only <code>user</code> and <code>assistant</code> roles are sent to the LLM
ProjectFile	Local record of an upload; <code>openai_file_id</code> set when stored via OpenAI Files API

Cascade deletes ensure removing a project removes all child records.

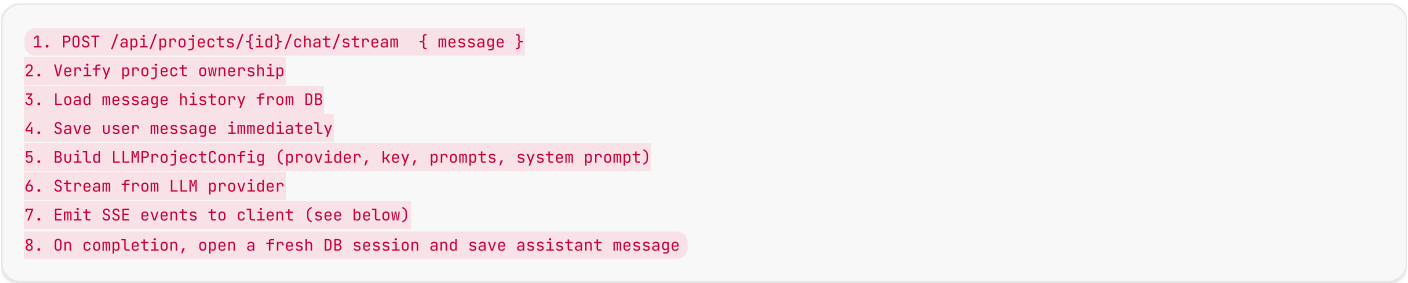
API keys are stored in the database and exposed only to the owning user on `GET /api/projects/{id}` and `PATCH`. List endpoints return `has_api_key: boolean` instead of the raw key.

Authentication Flow



Public routes: `POST /api/auth/register`, `POST /api/auth/login`. Everything else requires a valid token.

Chat Flow (Streaming)



SSE event protocol

The stream endpoint emits JSON payloads in standard SSE `data:` lines:

type	Meaning
status	Lifecycle hint: <code>connecting</code> → <code>thinking</code> → <code>streaming</code>
token	Partial assistant text chunk
done	Stream finished; includes saved <code>message</code> object
error	LLM or server error message

The frontend `api.chatStream()` async generator parses these events and updates the UI incrementally.

Instruction assembly

Before each LLM call, instructions are built from:

- 1. Project `system_prompt` (if set)
- 2. All prompt templates, formatted as `[name]\ncontent`

For **OpenAI**, instructions are passed to the Responses API. For **OpenRouter** and **custom** providers, they are prepended as a `system` message in chat/completions format.

LLM Integration

Provider	API path	Streaming
openai	OpenAI Responses API (<code>client.responses.create</code>)	Native SDK stream events (<code>response.output_text.delta</code>)
openrouter	<code>POST /chat/completions</code> via httpx	Manual SSE line parsing
custom	User-defined base URL + <code>/chat/completions</code>	Same as OpenRouter (Ollama, LM Studio, Z.AI, etc.)

Default base URLs:

- OpenAI — SDK default
- OpenRouter — `https://openrouter.ai/api/v1`
- Custom — must be set in project settings

Special handling: GLM/Z.AI models disable internal "thinking" traces so only user-visible `content` is streamed.

Missing API keys are rejected with `400` before any external call. Provider failures surface as `502` with a descriptive message.

File Uploads

Files are uploaded to `POST /api/projects/{id}/files` (max **20 MB**).

- **OpenAI provider:** file bytes are sent to the OpenAI Files API (`purpose=assistants`); `openai_file_id` is stored locally.
- **Other providers:** upload is rejected — file storage is OpenAI-specific today.

Deleting a file removes the local record only (no remote delete call).

Frontend Architecture

```
frontend/src/  
├── main.tsx  
├── App.tsx # React Router setup  
├── context/AuthContext.tsx # Global auth state, login/logout  
├── components/  
│   ├── ProtectedRoute.tsx # Redirect unauthenticated users  
│   ├── ChatMessageContent.tsx # Markdown rendering for messages  
│   └── ui/ # Button, Input, Select, Card, Textarea  
├── pages/  
│   ├── LoginPage.tsx  
│   ├── RegisterPage.tsx  
│   ├── DashboardPage.tsx # Agent list, create/delete  
│   └── ProjectPage.tsx # Tabbed agent workspace  
└── lib/
```

api.ts

utils.ts

Typed API client + SSE parser

Routing

Path	Access	Page
/login , /register	Public	Auth forms
/dashboard	Protected	Agent list
/projects/:id	Protected	Agent workspace

Project workspace tabs

ProjectPage organizes each agent into four tabs:

- **Chat** — message history, streaming input, clear history
- **LLM Settings** — provider, API key, base URL, model, system prompt
- **Prompts** — create/delete named prompt templates
- **Files** — upload and list files (OpenAI only)

Dev proxy

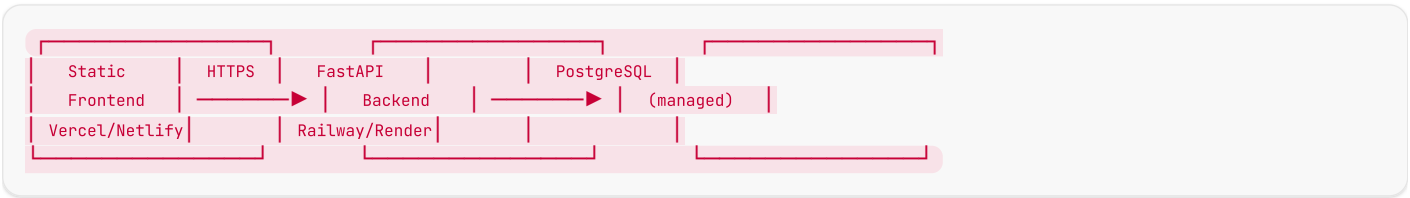
Vite proxies /api to http://localhost:8000 , so the frontend calls same-origin /api/ ... without CORS issues during development.

Security Model

Concern	Approach
Passwords	bcrypt via passlib; never stored in plaintext
Sessions	JWT only; no server session store
Authorization	Per-route ownership checks on user_id
API keys	Per-project, returned only to owner on detail endpoints
CORS	Configurable via CORS_ORIGINS env var
Upload size	20 MB hard limit

Production checklist: set a strong SECRET_KEY , use PostgreSQL, enable HTTPS, restrict CORS to your frontend origin.

Deployment Topology



Build the frontend (`npm run build`) and serve `dist/` from a CDN or static host. Run the backend with `uvicorn app.main:app` behind a process manager. Point `DATABASE_URL` at PostgreSQL.

Extension Points

Area	How to extend
<code>services/llm.py</code>	Add a provider branch or new adapter function
<code>routes/</code>	New modules for webhooks, analytics, teams
<code>models.py</code>	New entities (organizations, usage logs, vector stores)
<code>ProjectPage</code> tabs	New UI panels for RAG, tools, or integrations

Natural next steps

- Team/org support with role-based access
- RAG over uploaded documents (vector store + retrieval)
- Token usage tracking and rate limits per user/project
- OAuth social login
- Webhook integrations (Slack, Discord)
- Remote file delete and non-OpenAI file backends